

Node.js: primeiros passos com Express e TypeScript

Guia de conceitos, arquitetura de requisição e construção de APIs REST em memória.

Node.js LTS

Express 4

TypeScript

REST API

Este projeto é uma API REST simples para estudar os primeiros conceitos de Node.js usando Express e TypeScript. A aplicação mantém uma lista de tarefas em memória e disponibiliza operações para listar, criar, atualizar e remover itens.

CAMINHO COMPLETO DE UMA REQUISIÇÃO HTTP



1 Pré-requisitos

- **Node.js:** instalado (versão LTS recomendada).
- **npm:** gerenciador de pacotes padrão.
- **Cliente HTTP:** REST Client (VS Code), Insomnia ou Postman.
- **Conhecimentos:** Noções básicas de JS e TypeScript.

2 Como Executar

Instale as dependências e inicie o servidor:

```
npm install
npm run dev
```

O script usa `tsx watch` e disponibiliza a API em `http://localhost:3333`. Pare com `Ctrl+C`.

3 O que é Node.js?

Node.js é um ambiente de execução JavaScript server-side baseado no motor V8 do Google Chrome. Ele provê APIs nativas para manipulação de arquivos, redes e servidores HTTP. Neste projeto, o Node.js atua como runtime, enquanto o **Express** é a biblioteca responsável pelo roteamento e gestão de requisições.

4 O papel de cada arquivo

```
. |— package.json # Dependências, scripts e metadados do projeto |— tsconfig.json #
Configurações do compilador TypeScript |— src/ | |— server.tsx # Ponto de entrada e
inicialização do Express | |— routes.tsx # Definições de rotas, middlewares e handlers |—
build/ # Código JavaScript compilado (saída final) |— requisicoes.http # Arquivo de testes
de requisições HTTP
```

* Nota: Embora usem a extensão `.tsx`, não há componentes React. A extensão pode ser simplificada para `.ts` no futuro.

5 Configuração do Projeto

package.json

- `"type": "module"` : habilita ES Modules (`import/export`).
- `express` : framework web para rotas e HTTP.
- `tsx` : executa TypeScript diretamente sem build prévio.
- `@types/express` : definições de tipo para TS.

Script de desenvolvimento:

```
"dev": "CHOKIDAR_USEPOLLING=true tsx watch src/server.tsx"
```

tsconfig.json

- `rootDir: "./src"` : pasta do código fonte.
- `outDir: "./build"` : destino dos arquivos compilados.
- `module: "nodenext"` : resolução de módulos moderna do Node.
- `strict: true` : ativa verificação estrita de tipos.
- `noUncheckedIndexedAccess: true` : evita erros em acessos a índices de arrays.

6 Inicialização do Servidor (src/server.tsx)

O arquivo `server.tsx` configura e inicia a aplicação Express:

```
import express from "express";
import { router } from "../routes";

const app = express();
const PORT = 3333;

app.use(express.json());
app.use(router);

app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});
```

Fluxo de Execução:

1. `express()` instancia a aplicação.
2. `express.json()` interpreta requisições com corpo em JSON e popula `req.body`.
3. `app.use(router)` registra as rotas definidas.
4. `app.listen()` vincula o servidor à porta `3333`.

7 Router e Middleware (src/routes.tsx)

Separamos o roteamento e adicionamos middlewares para interceptar requisições:

Middleware Global (Log):

```
router.use((req, res, next) => {
  console.log("Passou no Mid Global");
  return next();
});
```

Middleware de Validação (checkNome):

```
function checkNome(req: Request,
  res: Response, next: NextFunction) {
  if (!req.body.nome) {
    return res.status(400)
      .json({ message: "Nome é obrigatório" });
  }
}
```

8 Parâmetros de uma Requisição

Query Params

Enviados na URL após o símbolo

?

Ex: `/tarefas?nome=Estudar`

Acesso: `req.query.nome`

Route Params

Parâmetros identificadores na rota.

Ex: `/tarefas/:index`

Acesso: `req.params.index`

Request Body

Corpo de dados no formato JSON.

Ex: `{ "nome": "Comprar pão" }`

Acesso: `req.body`

9 Endpoints Implementados

MÉTODO	ENDPOINT	OBJETIVO
GET	<code>/api/tarefas</code>	Retorna a lista completa de tarefas.
GET	<code>/api/tarefas/:index</code>	Retorna a tarefa específica pelo índice do array.
POST	<code>/api/tarefas</code>	Adiciona uma nova tarefa (requer <code>checkNome</code>).
PUT	<code>/api/tarefas/:index</code>	Substitui o nome da tarefa no índice especificado.
DELETE	<code>/api/tarefas/:index</code>	Remove a tarefa do array no índice fornecido.

Exemplos de Chamadas e Respostas HTTP:

```
# GET /api/tarefas
HTTP/1.1 200 OK
[
  "Comprar Pao",
  "Lavar Roupa",
  "Estudar Node.js"
]
```

```
# POST /api/tarefas
Content-Type: application/json

{ "nome": "Revisar Express" }
```

```
# GET /api/tarefas/1
HTTP/1.1 200 OK
"Lavar Roupa"
```

```
# DELETE /api/tarefas/1
HTTP/1.1 200 OK
# Executa tarefas.splice(index, 1)
```

10 Como Testar

Utilize o arquivo `requisicoes.http` com a extensão REST Client do VS Code na seguinte ordem recomendada:

1. Listar tarefas (`GET`).
2. Buscar tarefa por índice (`GET /api/tarefas/1`).
3. Criar tarefa com `nome` preenchido (`POST`).
4. Atualizar (`PUT`) e remover (`DELETE`).
5. Testar `POST` com corpo vazio para validar o retorno de erro `400 Bad Request` .

11 Estado em Memória e Limitações

Atenção: A lista de tarefas é mantida em um array JavaScript local (`const tarefas = [...]`).

- As alterações são perdidas ao reiniciar o servidor.
- Não há persistência em Banco de Dados nem autenticação.
- A remoção de itens altera a posição (índices) dos elementos restantes.

12 Roteiro de Estudo

1. Execute o servidor e observe a porta.
2. Analise o arquivo `server.tsx` .
3. Entenda como o `router` se conecta ao app.
4. Acompanhe o fluxo do middleware e `next()` .
5. Pratique chamadas com diferentes verbos HTTP.
6. Compare `req.query` , `params` e `body` .
7. Adicione tratamento para rotas inexistentes (`404`).

13 Próximas Melhorias

- Adicionar scripts de build e start para produção.
- Renomear extensões de `.tsx` para `.ts` .
- Validar se o índice buscado existe no array.
- Retornar `404 Not Found` em tarefas inexistentes.
- Separar lógica em Controllers e Services.
- Conectar a um banco de dados (SQLite/PostgreSQL).
- Implementar rotas de usuários e autenticação JWT.